Who Wrote It? Identifying Large Language Models from Their Responses

Advait Sadineni (team leader and sole member)
CMPSC 448, The Pennsylvania State University
October 11, 2026 · code and data: github.com/REPO_URL

Abstract. Can a classifier tell which large language model wrote a response? I treat LLM attribution as supervised text classification and study it on two datasets: 20,000 responses from five 2023 model families drawn from UltraFeedback with an identical prompt mix per family, and a new set I generated in which 1,118 prompts were each answered by five 2026 families (Nemotron 3, gpt-oss, Kimi K3, Llama 3.2, DiffusionGemma). The two required models, a TextCNN and a BiLSTM, reach 0.736 and 0.725 macro-F1 on the 2023 families (chance 0.20) and 0.852 and 0.819 on the 2026 families; a fine-tuned DistilBERT adds under two points, and an ensemble reaches 0.781 and 0.860. The user's prompt carries no information about the model once prompt assignment is controlled, and prompt-only accuracy above chance in uncontrolled data is shown to be an artefact of how prompts were distributed. Fingerprints transfer across task domains except to one-word-answer tasks, need only a few hundred examples per model, and do not survive a model generation: a classifier trained on Llama-2 labels 2% of Llama 3.2 responses as Llama. Ablations, filter probing and saliency show the fingerprint is style rather than content: keeping only function words, punctuation and line breaks does not hurt the BiLSTM at all, and on 2026 models a tenth of the accuracy comes from typographic choices such as non-breaking hyphens and em dashes. A paraphrase attack quantifies how much of the signal survives rewriting. All code, data and a browser demo are public.

1 Introduction and problem definition

Large language models now write a large share of the text people read, and different models are trained on different data with different post-training recipes. If those differences leave a measurable trace in the text, then LLM output is attributable: given a response, a classifier can say which model family produced it. This project treats attribution as supervised text classification and asks four questions.

- **RQ1.** Given only a response, can we identify the model family that generated it?
- **RQ2.** Does the user's prompt help, and if prompt-only classification works, why?
- **RQ3.** Do the learned fingerprints generalise across task domains, across model generations, and how much data do they need?
- **RQ4.** What are the fingerprints made of: content, style, or formatting?

The required models are a convolutional network (TextCNN) and a recurrent network (BiLSTM). I add a classical TF-IDF baseline below them and a fine-tuned DistilBERT above them, plus two hybrids, so every number has context. Every headline result is the mean of three random seeds, every split is grouped by prompt so no prompt leaks between training and test, and every table cell in this report is produced by one command in the repository.

2 Data

2.1 Dataset A: UltraFeedback (2023 models)

UltraFeedback [1] is a public, MIT-licensed preference dataset in which 63,967 instructions from six sources were each answered by four models sampled from a pool of seventeen. Because it stores the model name next to every completion, it is exactly the `(LLM_name, LLM_input, LLM_output)` triple the project asks for, and because each prompt was answered by several models it supports controlled comparisons. I flattened the 256k completions, dropped empty, very short (under 20 characters) and exact-duplicate responses, and kept five clearly distinct model families: **GPT** (gpt-4, gpt-3.5-turbo), **Llama-2-chat** (7B, 13B, 70B), **Bard**, **Falcon-40B-instruct** and **MPT-30B-chat**. Llama fine-tunes in the pool (WizardLM, Vicuna, UltraLM, Alpaca) share Llama's base weights and were excluded from the family task; they return in the 16-way fine-grained experiment.

Controlled source mix. The six prompt sources (Flan, ShareGPT, Evol-Instruct, UltraChat, Truth-

Table 1: The four datasets. All splits are grouped by prompt; `uf` and `uf_natural` draw from disjoint random samples of the same pool.

Dataset	Classes	Rows (train / val / test)	Purpose
`uf`	5 families	20,000 (16,015 / 1,991 / 1,994)	main; identical source mix per family
`uf_natural`	5 families	20,000 (15,983 / 2,005 / 2,012)	natural source mix; RQ2 confound probe
`uf17`	16 models	24,000 (19,219 / 2,375 / 2,406)	fine-grained: can we separate Llama-2-7B from 70B?
`modern`	5 families	5,590 (4,470 / 560 / 560)	2026 models; 1,118 prompts, every one answered by all five families

fulQA, FalseQA) differ in style and length, and the families are not equally represented in each source (Bard answered only 701 Flan prompts, GPT-4 answered 5,456). A classifier could therefore learn "this prompt looks like Flan, Flan prompts are rarely Bard" instead of a model fingerprint. To rule this out, the main dataset (`uf`) samples **exactly the same number of responses per source for every family**: 1,000 Evol-Instruct, 1,000 ShareGPT, 800 UltraChat, 600 Flan, 450 FalseQA and 150 TruthfulQA, for 4,000 responses per family and 20,000 in total. A second variant (`uf_natural`) draws 4,000 per family with each family's natural source mix, and is used only to show what happens when the confound is left in (RQ2).

Grouped split. Prompts, not responses, are assigned to train, validation and test (80/10/10), so the 2–4 responses to one prompt always land in the same split. Without this, a model could memorise prompt-specific content. The split script asserts zero prompt overlap and the per-source balance of every split.

Confound worth stating. UltraFeedback generated each completion under a randomly chosen system prompt (a "principle" such as honesty or helpfulness). This adds within-family variance; it does not favour any family.

2.2 Dataset B: a modern set I generated (2026 models)

The 2023 pool does not contain today's models, and in UltraFeedback a given prompt was answered by only four of the seventeen models. I therefore built a second dataset with a complete block design: the **same** 1,200 prompts, sampled from the UltraFeedback **test** prompts with the same source quotas, were each sent to five current model families through NVIDIA's OpenAI-compatible endpoint (`integrate.api.nvidia.com`): **Nemotron-3** (nvidia/nemotron-3-super-120b-a12b), **gpt-oss** (openai/gpt-oss-20b), **Kimi K3** (moonshotai/kimi-k3), **Llama 3.2** (meta/llama-3.2-11b-vision-instruct) and **DiffusionGemma** (google/diffusiongemma-26b-a4b-it, a non-autoregressive diffusion language model). Sampling used temperature 0.7 and a 450-token visible-answer budget (reasoning models received a larger budget for hidden reasoning, which was discarded); the classifiers read at most 512 tokens anyway, and truncation applies equally to every family. Generation took about eight hours of wall-clock time at the endpoint's 40 requests-per-minute free-tier limit, most of it spent discovering which models the endpoint would actually serve at a usable rate. Three further families (Gemma 4 31B, GLM 5.3, Llama 3.2 90B) were started and abandoned because the endpoint queued them at under one response per minute; DeepSeek v4.1 never returned within four minutes. Only prompts answered by every family are kept, so prompt-only classification is at chance by construction, which is the clean control RQ2 needs. The prompts are public; nothing private was sent to the API.

3 Models and training

3.1 Text representation

The tokenizer is a regular expression that keeps **case**, **punctuation** and **newlines** as tokens. This is deliberate: a lower-casing, punctuation-stripping tokenizer would erase exactly the signals (markdown bullets, bold markers, capitalised openers, line breaks) that RQ4 shows the models rely on. The vocabulary is the 30,000 most frequent training tokens with frequency at least two; the rest map to `<unk>`. Responses are truncated to 512 tokens, prompts to 128, and the input+output view concatenates them around a `<sep>` token (640 tokens). The Char-CNN reads the first 1,024 characters with a 97-symbol alphabet instead.

3.2 The six classifiers

Table 2: The model ladder. The two required models sit in the middle; the baseline below and the transformer above give them context.

Model	Architecture	Params
TF-IDF + LR	word 1–2-grams and character 3–5-grams (200k features each, sublinear tf, no lower-casing) → multinomial logistic regression, $C = 5$	—
TextCNN (required)	embedding 128 → three parallel 1-D convolutions (widths 3, 4, 5; 128 channels each) → ReLU → global max-pool → dropout 0.5 → linear [2]	4.04 M
BiLSTM (required)	embedding 128 → bidirectional LSTM (128 hidden per direction) → concatenation of masked max-pool and mean-pool → dropout 0.5 → linear	4.10 M
CNN-BiLSTM	embedding 128 → width-3 convolution (128 ch) → BiLSTM (128) → max-pool → linear; the "combine models" variant	4.15 M
Char-CNN	character embedding 32 → six 1-D convolutions (256 ch, widths 7,7,3,3,3,3 with three max-pools of 3) → 512-unit dense → linear [3]	5.77 M
DistilBERT	`distilbert-base-cased` fine-tuned end to end, 512-token input, linear head; cased so capitalisation survives	66 M

3.3 Training details

All neural models are trained in PyTorch 2.11 on one RTX 5080 laptop GPU with bf16 autocast. TextCNN, BiLSTM, CNN-BiLSTM and Char-CNN use AdamW (learning rate 10^{-3}, weight decay 10^{-2}), batch size 64, gradient clipping at 1.0, cross-entropy loss, at most 15 epochs with early stopping on validation macro-F1 (patience 3) and the best-validation checkpoint restored. DistilBERT uses AdamW at $2 \cdot 10^{-5}$ with 6% linear warm-up then linear decay, batch 16, three epochs. Embeddings are learned from scratch (no pretrained vectors), which keeps the required models honest about what they learn from 16k examples. One TextCNN run takes about 25 s, one BiLSTM run about 45 s, DistilBERT about 20 minutes (sharing the GPU with other runs). Every configuration in Sections 4 and 5 was run with seeds 42, 1 and 2; tables report mean ± standard deviation. The metric is macro-F1 (classes are balanced, so accuracy is nearly identical and is reported alongside).

3.4 Hyper-parameter selection (validation only)

The defaults above are the textbook settings. To check they are not leaving accuracy on the table, I ran a small grid for each required model on the **validation** split of `uf` (seed 42, output view): TextCNN over embedding size {128, 300} × channels {128, 256} × filter widths {(3,4,5), (2,3,4,5)}, and BiLSTM over embedding {128, 300} × hidden {128, 256} × layers {1, 2}, sixteen runs in all (Table 3). The rule, fixed before the sweep ran, was: adopt a configuration as "tuned" only if its validation macro-F1 beats the default by at least 0.01; the test set is never used for selection. Both models cleared the bar. The tuned TextCNN uses 300-dimensional embeddings and adds a width-2 filter (validation 0.750 vs 0.733); the tuned BiLSTM uses 300-dimensional embeddings and 256 hidden units per direction (0.763 vs 0.726). Larger embeddings help both; a second LSTM layer and more CNN channels do not. The tuned models were then retrained with three seeds and appear as separate rows in the result tables, so the default and tuned numbers can be compared directly.

Table 4: RQ1 and RQ2 on the controlled UltraFeedback set (5 families, test n = 1,994). Mean ± std over seeds 42, 1, 2; DistilBERT and TF-IDF are single runs (TF-IDF is deterministic). Chance is 0.20. Bold marks the best model per view.

Model	Output acc.	Output F1	Input acc.	Input F1	Both acc.	Both F1
TF-IDF + LR	0.709	0.710	0.219	**0.219**	0.683 ± 0.005	0.682 ± 0.005
TextCNN	0.733 ± 0.006	0.736 ± 0.007	0.216 ± 0.003	0.212 ± 0.002	0.710 ± 0.005	0.710 ± 0.004
BiLSTM	0.722 ± 0.010	0.725 ± 0.009	0.217 ± 0.004	0.214 ± 0.006	0.699 ± 0.003	0.702 ± 0.003
TextCNN (tuned)	0.740 ± 0.007	0.741 ± 0.007	—	—	—	—
BiLSTM (tuned)	0.748 ± 0.004	0.751 ± 0.002	—	—	—	—
CNN-BiLSTM	0.725 ± 0.016	0.728 ± 0.013	0.226 ± 0.003	0.205 ± 0.018	0.713 ± 0.009	0.714 ± 0.009
Char-CNN	0.593 ± 0.012	0.602 ± 0.013	0.199 ± 0.005	0.068 ± 0.003	0.557 ± 0.048	0.556 ± 0.046
DistilBERT	0.755 ± 0.003	0.756 ± 0.003	0.231	0.216	0.747	**0.749**
Ensemble (TF-IDF+CNN+LSTM+BERT)	0.779 ± 0.004	**0.781 ± 0.003**	—	—	—	—

Table 3: Validation macro-F1 for every configuration in the sweep (seed 42). Defaults: TextCNN emb 128 / 128 channels / widths 3,4,5; BiLSTM emb 128 / hidden 128 / 1 layer.

Model	Configuration	Val macro-F1
TextCNN	emb = 300, channels = 128, kernels = 2,3,4,5	0.750
TextCNN	emb = 300, channels = 256, kernels = 3,4,5	0.749
TextCNN	emb = 300, channels = 256, kernels = 2,3,4,5	0.749
TextCNN	emb = 300, channels = 128, kernels = 3,4,5	0.742
TextCNN	emb = 128, channels = 256, kernels = 2,3,4,5	0.737
TextCNN	emb = 128, channels = 128, kernels = 3,4,5	0.733
TextCNN	emb = 128, channels = 128, kernels = 2,3,4,5	0.725
TextCNN	emb = 128, channels = 256, kernels = 3,4,5	0.717
BiLSTM	emb = 300, hidden = 256, layers = 1	0.763
BiLSTM	emb = 300, hidden = 256, layers = 2	0.752
BiLSTM	emb = 300, hidden = 128, layers = 2	0.748
BiLSTM	emb = 128, hidden = 256, layers = 1	0.746
BiLSTM	emb = 128, hidden = 256, layers = 2	0.745
BiLSTM	emb = 128, hidden = 128, layers = 2	0.737
BiLSTM	emb = 128, hidden = 128, layers = 1	0.726
BiLSTM	emb = 300, hidden = 128, layers = 1	0.724

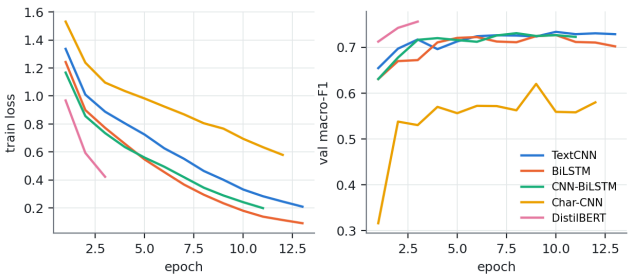


Figure 1: Training loss and validation macro-F1 per epoch (seed 42, output view, `uf`). TextCNN keeps improving on validation long after the LSTM plateaus; both overfit the training set within ten epochs.

4 Results: RQ1 and RQ2

4.1 RQ1: can we identify the model from its response?

Yes, and comfortably. On the controlled five-family UltraFeedback task, every classifier is far above the 0.20 chance level when it sees only the response (Table 4, "Output" columns; Figure 2). The required TextCNN reaches 0.736 ± 0.007 macro-F1 over three seeds and the required BiLSTM 0.725 ± 0.009. The TF-IDF baseline is only a few points behind at 0.710, the CNN-BiLSTM hybrid sits between its two parents, and the character-level CNN is the weakest neural model at 0.602. Fine-tuning DistilBERT, with sixteen times the parameters and pretraining on billions of words, buys 0.753: less than two points over a CNN with randomly initialised embeddings that trains in 25 seconds.

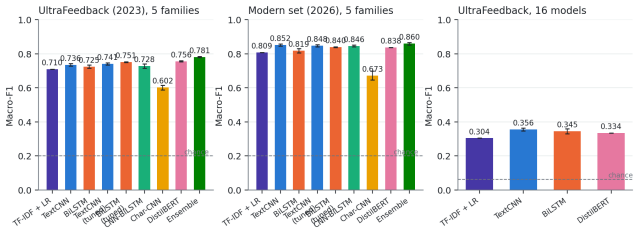


Figure 2: Macro-F1 of each classifier when it sees only the response. Error bars are one standard deviation over three seeds.

TextCNN beats BiLSTM, and the difference is real. On the seed-42 test set the CNN is right on 210 responses the LSTM gets wrong, and the LSTM is right on 154 the CNN gets wrong; an exact McNemar test gives $p = 0.004$. The two models disagree on about 18% of responses, so they are not interchangeable, which is why the hybrid does not simply inherit the better of the two. The CNN also trains three times faster per epoch (2 s vs 3 s at batch 64) and keeps improving on the validation set for longer (Figure 8 in Section 3).

Which families are easy? Table 5 and the confusion matrices in Figure 3 tell the same story for every model: Bard (F1 0.86) and Llama-2 (0.84) are the most recognisable, GPT and Falcon are in the middle (0.72), and MPT-30B is the hardest (0.58). MPT's errors go to Falcon and GPT in roughly equal measure, and in the fine-grained experiment of Section 5 it is the least identifiable of all sixteen models; its responses are the least stylistically committed of the five. Llama-2 is rarely confused with anything: its chat tuning produces a very consistent opener ("Hello! I'm here to help you with your question") that the n-gram probe in Section 5 recovers almost verbatim.

Table 5: Per-family F1 on the controlled UltraFeedback test set (seed 42, output view).

Model	Bard	Falcon	GPT	Llama-2	MPT
TextCNN	0.86	0.72	0.72	0.84	0.58
BiLSTM	0.83	0.68	0.68	0.85	0.54
DistilBERT	0.87	0.75	0.71	0.85	0.59

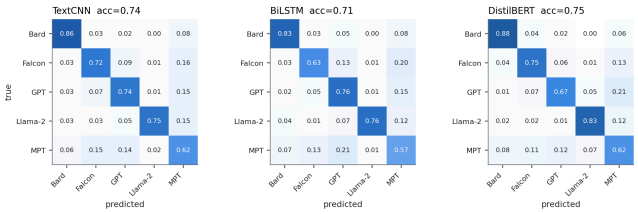


Figure 3: Row-normalised confusion matrices, output view, seed 42. The MPT row is the only one with large off-diagonal mass.

4.2 RQ2: does the prompt help?

Prompt alone: no. With the input view, in which the classifier sees only the user's prompt, every model scores 0.21–0.22 macro-F1, indistinguishable from chance, and the Char-CNN collapses to predicting one class (F1 0.07). This is what the dataset was built to make true: each family answers exactly the same number of prompts from each source, and UltraFeedback assigned models to prompts at random, so there is no information about the model in the prompt. A "source oracle" that predicts the majority family of each prompt source from the training split confirms it: 18.5% accuracy on the test set.

Prompt plus response: slightly worse than response alone. Concatenating the prompt in front of the response ("Both" columns) costs every neural model one to three points. This is not a bug in the data; it is capacity. The prompt occupies up to 128 of the 640 tokens with text that carries no label information, the vocabulary now has to cover prompt words, and the max-pooled features are diluted. DistilBERT, which can attend selectively, loses only 0.4 points. The practical answer to the project's question is therefore: the prompt does not help identify the model, and a classifier that is given it has to learn to ignore it.

When input-only "works", it is the dataset talking. The `uf_natural` variant keeps each family's natural mix of prompt sources, so Bard is over-represented on UltraChat and Evol-Instruct prompts and under-represented on Flan. Now the input-only classifiers climb to 0.23–0.24 (Table 6), and the source oracle alone reaches 28.0% accuracy. The classifier has learned nothing about the models; it has learned which kinds of prompts each model happened to be asked. This is the mechanism behind question (iii) in the project description: in any dataset where models were not asked the same prompts, prompt-only accuracy above chance measures the sampling procedure, not the model. The modern set in Section 5, where every family answered the identical 1,200 prompts, is the cleanest version of this control.

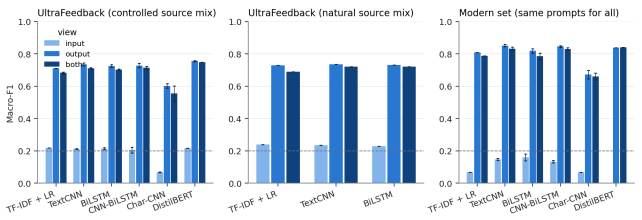


Figure 4: Input, output and input+output views across datasets. The dashed line is chance.

Table 6: The same three views on `uf_natural`, where each family keeps its natural prompt-source distribution. Input-only rises above chance only because prompt source is now correlated with family (source-oracle accuracy 28.0% vs 18.5% on the controlled set).

Model	Output acc.	Output F1	Input acc.	Input F1	Both acc.	Both F1
TF-IDF + LR	0.727	0.729	0.243	**0.240**	0.690	0.690
TextCNN	0.731	**0.735**	0.255	0.235	0.719	**0.722**
BiLSTM	0.724	0.732	0.229	0.228	0.716	0.722

Table 7: RQ1 and RQ2 on the modern set (5 families, test n = 560; chance 0.20). Mean ± std over three seeds; DistilBERT and TF-IDF single runs. Input-only accuracy is exactly 0.200 for every model because every prompt was answered by all five families.

Model	Output acc.	Output F1	Input acc.	Input F1	Both acc.	Both F1
TF-IDF + LR	0.807	0.809	0.200	0.067	0.786	0.788
TextCNN	0.851 ± 0.007	0.852 ± 0.007	0.200	0.147 ± 0.007	0.829 ± 0.008	0.833 ± 0.009
BiLSTM	0.817 ± 0.013	0.819 ± 0.013	0.200	**0.160 ± 0.021**	0.782 ± 0.020	0.786 ± 0.020
TextCNN (tuned)	0.847 ± 0.006	0.848 ± 0.007	—	—	—	—
BiLSTM (tuned)	0.838 ± 0.001	0.840 ± 0.002	—	—	—	—
CNN-BiLSTM	0.845 ± 0.005	0.846 ± 0.005	0.200	0.133 ± 0.008	0.826 ± 0.008	0.829 ± 0.008
Char-CNN	0.673 ± 0.029	0.673 ± 0.026	0.200	0.067	0.662 ± 0.023	0.660 ± 0.021
DistilBERT	0.838	0.838	—	—	0.839	**0.841**
Ensemble (TF-IDF+CNN+LSTM+BERT)	0.858 ± 0.008	**0.860 ± 0.008**	—	—	—	—

4.3 The same questions on 2026 models

The modern set repeats RQ1 and RQ2 on five current families with the cleanest possible design: every one of the 1,118 prompts was answered by all five models. The answers are sharper on both questions (Table 7, Figure 5).

RQ1. Today's models are **easier** to tell apart than the 2023 ones. The TextCNN reaches 0.852 ± 0.007 macro-F1 (five-way, chance 0.20) on 4,470 training responses, a quarter of the UltraFeedback training set, and the BiLSTM 0.819 ± 0.013; tuning narrows the gap again (0.848 vs 0.840). DistilBERT scores 0.838, below the CNN, and the four-model ensemble 0.860 ± 0.008. gpt-oss is the most recognisable family (per-class F1 0.93), DiffusionGemma the least (0.83); the main confusions run between Llama 3.2 and the two NVIDIA-hosted reasoning models. The CNN again beats the LSTM at default settings (McNemar $p = 0.04$), and the difference between the ensemble and the CNN alone is not significant ($p = 0.29$): on this data one well-chosen small model is essentially as good as everything combined.

RQ2. With identical prompts for every family the input-only view is at chance by construction and the classifiers confirm it: accuracy exactly 0.200, macro-F1 between 0.07 and 0.16 as the models collapse to one or two classes. Adding the prompt to the response costs 2–3 points for the CNN and LSTM, as on UltraFeedback. The combination of this result with Table 6 settles the project's question (iii): prompt-only accuracy above chance is a property of how a dataset assigned prompts to models, never of the models.

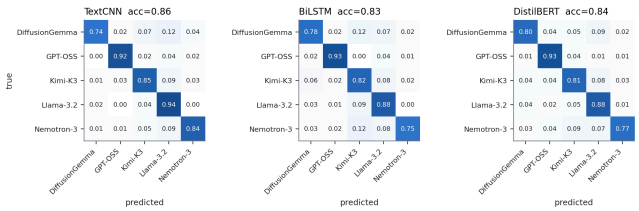


Figure 5: Row-normalised confusion matrices on the modern test set, seed 42.

Why are the 2026 models easier? The stylometry in Figure 6 says most of it: the modern families differ enormously in **how much** they write and **how they format it**. gpt-oss writes a median of 500 words with 40 line breaks and thirteen bold spans per response; DiffusionGemma, Llama 3.2 and Nemotron-3 write about 250 words; Llama 3.2 almost never uses markdown headers (7% of responses, against 47–69% for the others) and is the only modern family that still hedges ("It's important to note", 0.32 per response). A gradient-boosted classifier on the 26 stylometric features alone reaches 71.4% accuracy on the modern set, against 64.1% on UltraFeedback.

Typographic fingerprints, and a data-quality caveat. Probing the modern CNN's filters (the method of Section 5.3) surfaces signals that are real but almost embarrassingly shallow (Table 8). 79% of gpt-oss responses contain a Unicode **non-breaking hyphen** (U+2011, as in "step-by-step"), which no other fam-

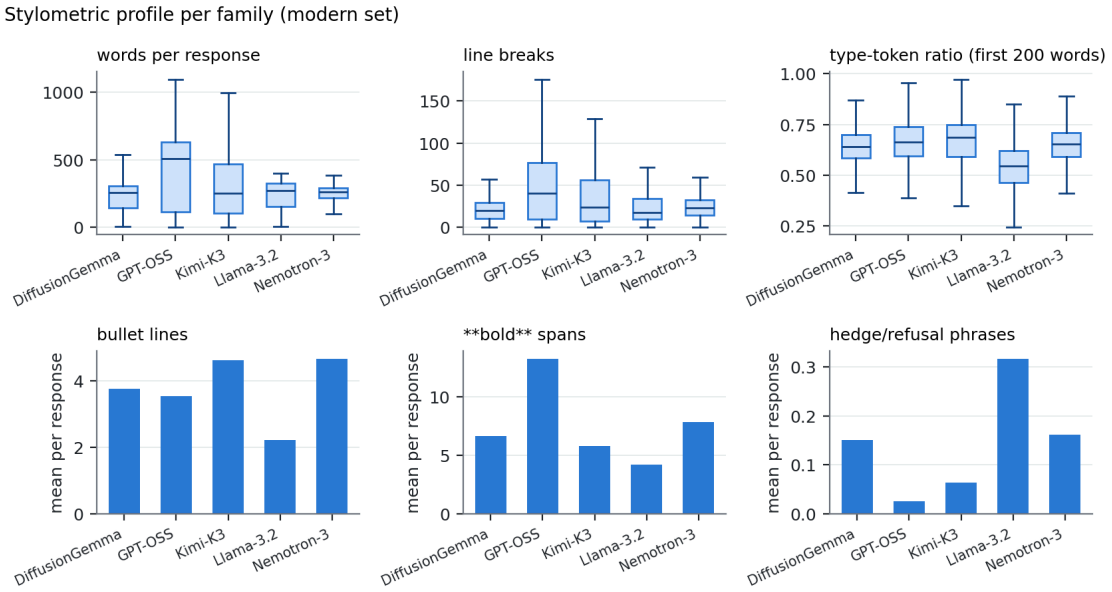


Figure 6: Stylometric profile of the five 2026 families. Formatting intensity, not vocabulary, separates them.

Table 8: Share of modern responses containing each formatting or typographic pattern. The 2023 equivalents are in the appendix (Table 15).

Pattern	DiffusionGemma	GPT-OSS	Kimi-K3	Llama-3.2	Nemotron-3
Markdown header	0.65	0.60	0.47	0.07	0.69
Horizontal rule (three hyphens)	0.27	0.51	0.13	0.01	0.54
Bold (double asterisks)	0.83	0.85	0.72	0.54	0.85
En dash U+2013	0.08	0.60	0.20	0.01	0.25
Em dash U+2014	0.16	0.28	0.57	0.00	0.72
Non-breaking hyphen U+2011	0.00	0.79	0.00	0.00	0.00
Template token <|…|>	0.00	0.00	0.17	0.00	0.00

ily emits; 72% of Nemotron-3 and 57% of Kimi K3 responses contain an em dash, against 0.1% for Llama 3.2; Nemotron-3 and gpt-oss separate paragraphs with `---` rules half the time; and 17% of Kimi K3 responses contain a leaked chat-template token such as `<|close|>`, usually at the head of a degenerate tail of text, which is a defect of that endpoint's serving stack rather than a property of the model. These are exactly the "signals the classifier appears to use" that RQ4 asks about, so I kept them in the data and measured their contribution instead: retraining with template tokens removed and all typographic dashes and quotes folded to ASCII (Table 9) costs the TextCNN 8.3 points (0.852 → 0.769 ± 0.018) and the BiLSTM 7.4 (0.819 → 0.744 ± 0.015). Even without them both required models stay far above chance and above their UltraFeedback scores, so the modern fingerprint is not **only** punctuation; but nearly a tenth of the accuracy on 2026 models comes from characters a copy-editor would normalise away. Any deployed attributor should be trained on ASCII-folded text, or it will be defeated by a find-and-replace.

Table 9: Modern set, output view, three seeds: effect of removing leaked template tokens and ASCII-folding typographic punctuation before training.

Model	Full text	Template tokens removed, punctuation ASCII-folded	Δ
TextCNN	0.852 ± 0.007	0.769 ± 0.018	−0.083
BiLSTM	0.819 ± 0.013	0.744 ± 0.015	−0.074

5 In-depth analyses

5.1 RQ3a: do fingerprints transfer across task domains?

Each classifier was retrained six times on five prompt sources and tested on every response from the sixth, which it had never seen (Table 10, Figure 7). Fingerprints transfer, with a cost that depends on how different the held-out domain is. Holding out ShareGPT, UltraChat or Evol-Instruct (open-ended chat and in-

Table 10: Leave-one-source-out macro-F1 on UltraFeedback (seed 42). Each column is a classifier trained on the other five sources and tested on all responses from the named source.

Model	In-domain	evol-instruct	false-qa	flan	sharegpt	truthful-qa	ultrachat	Mean held-out
TF-IDF + LR	0.710	0.670	0.650	0.543	0.684	0.700	0.710	0.660
TextCNN	0.736	0.728	0.627	0.557	0.714	0.687	0.715	0.671
BiLSTM	0.725	0.666	0.652	0.561	0.691	0.646	0.694	0.652

Table 11: Leave-one-source-out macro-F1 on the modern set (seed 42).

Model	In-domain	evol-instruct	false-qa	flan	sharegpt	truthful-qa	ultrachat	Mean held-out
TF-IDF + LR	0.809	0.839	0.862	0.440	0.879	0.890	0.894	0.801
TextCNN	0.852	0.851	0.916	0.420	0.892	0.913	0.892	0.814
BiLSTM	0.819	0.786	0.839	0.353	0.831	0.873	0.859	0.757

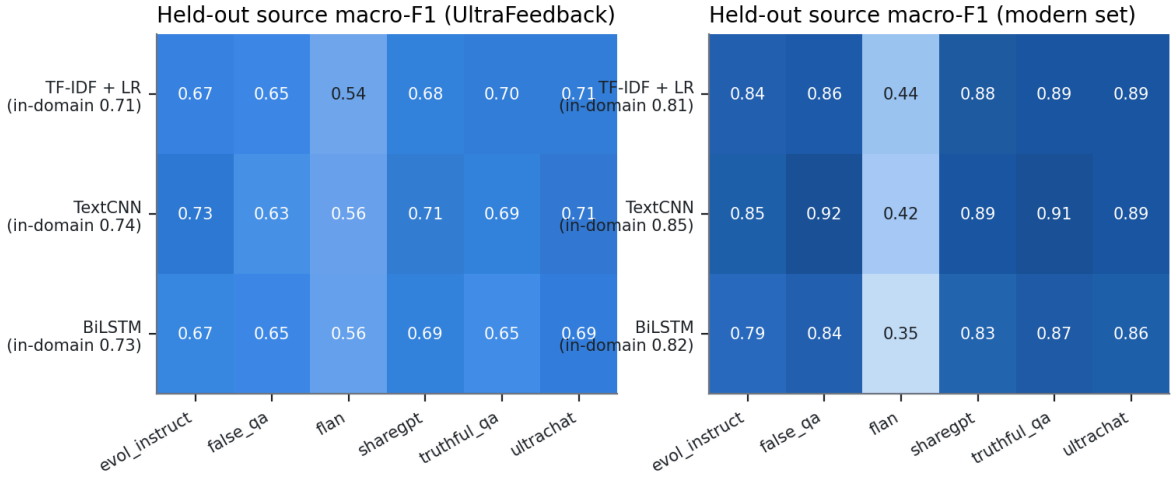


Figure 7: Held-out-source macro-F1 on both datasets. Flan is the hard domain for every model.

struction following, the bulk of the data) costs the CNN two to three points. Holding out TruthfulQA costs five. Holding out Flan, whose prompts are short, templated NLP tasks that often demand a one-word answer, costs eighteen points (0.736 → 0.557): when a model is asked to output "yes" or a single entity there is little room for style, and the stylistic cues the classifier learned elsewhere (openers, list formatting, sign-offs) are absent. Falcon and MPT suffer most on held-out Flan; Llama-2 and Bard remain identifiable everywhere, consistent with their per-class F1 in-domain. So the classifier is learning something closer to a general fingerprint than to task-specific patterns, but the fingerprint lives in the parts of a response that short-answer tasks suppress.

The modern set tells the same story more sharply (Table 11, right panel of Figure 7). Held-out ShareGPT, UltraChat, TruthfulQA and FalseQA are classified **at least as well** as in-domain (0.86–0.92 for the CNN against 0.85 in-domain, partly because the held-out test is larger and the in-domain test contains Flan), while held-out Flan drops to 0.42 for the CNN and 0.35 for the LSTM. The 2026 fingerprint is formatting-heavy, and a one-line answer has no formatting.

5.2 RQ3b: how much data does a fingerprint need?

Table 12: Learning curve, output view, mean ± std over three seeds. The full training set is 16,015 responses.

Training examples	TextCNN F1	BiLSTM F1
801 (5%)	0.567 ± 0.032	0.492 ± 0.067
1,602 (10%)	0.618 ± 0.019	0.576 ± 0.012
4,004 (25%)	0.701 ± 0.008	0.634 ± 0.008
8,008 (50%)	0.719 ± 0.007	0.674 ± 0.018
16,015 (100%)	0.736 ± 0.007	0.725 ± 0.009

With only 800 training responses (160 per family) the CNN already reaches 0.567 and the LSTM 0.492; with 4,000 the CNN is within four points of its final score (Table 12, Figure 8). The CNN is consistently more data-efficient than the LSTM, by seven points at 5% of the data, which fits the ablation findings below: the signal is in short local patterns that a few hundred examples suffice to reveal, and an LSTM has to learn

to carry them through long sequences before it can use them.

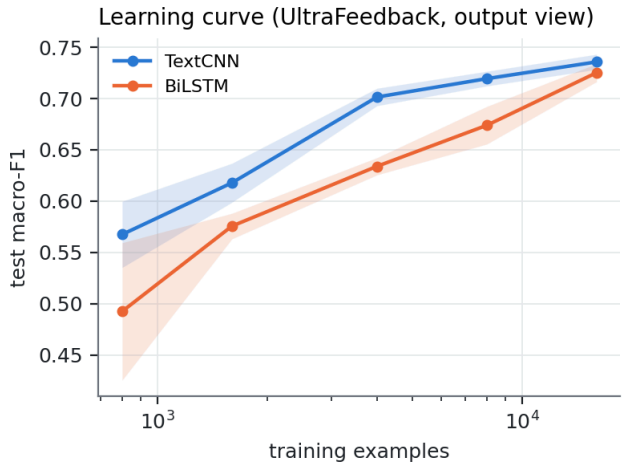


Figure 8: Test macro-F1 against training-set size (log scale). Shaded bands are one standard deviation over seeds.

5.3 RQ3c: do fingerprints survive a model generation?

If a fingerprint belonged to a lineage, a classifier that learned Llama-2-chat in 2023 should recognise Llama 3.2 in 2026. It does not (Figure 9). Applied to the 5,590 modern responses, the UltraFeedback TextCNN labels only 1.9% of Llama 3.2′s responses as Llama-2; it calls them GPT (39%), Bard (32%) or MPT (23%) instead. The other modern families are sorted almost entirely into the two 2023 families that used markdown most, GPT and Bard: Nemotron-3 is "GPT" 74% of the time, DiffusionGemma "Bard" 66%. The classifier is confident while doing this (mean max-probability 0.80), which is the usual failure mode of a discriminative model under distribution shift.

Together with the fine-grained result (Llama-2 sizes separable, GPT-3.5 and GPT-4 not) this says what a fingerprint is: not the base model's weights or lineage, but the post-training recipe of a particular release. Llama-2-chat's "Hello! I'm here to help" is gone from Llama 3.2; what remains of Llama 3.2′s identity in 2026 is that it formats less than its contemporaries. Attribution systems therefore need to be re-enrolled for every model release, which the learning curve in Table 12 suggests is cheap: a few hundred examples per model.

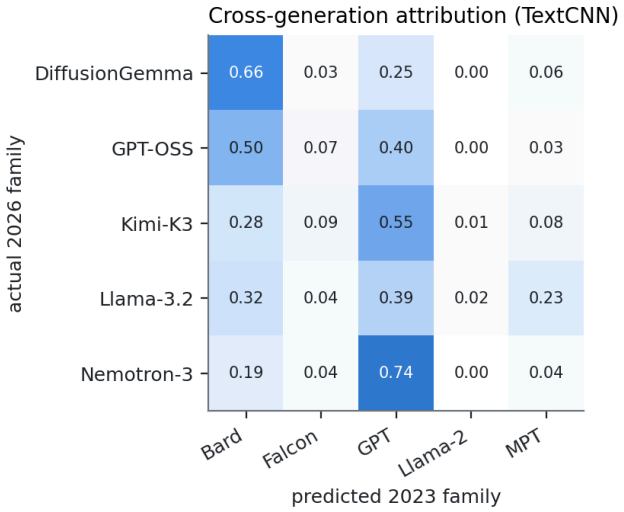


Figure 9: Where the 2023 classifier puts 2026 responses. Rows are actual modern families, columns the predicted UltraFeedback family (row-normalised).

5.4 RQ4: what is the fingerprint made of?

Stylometry. Twenty-six hand-computed features per response (length, sentence statistics, type-token ratio, line breaks, markdown markers, hedges, punctuation) differ sharply across families (Figure 10). Falcon writes the shortest responses (median 98 words against 196–262 for the others) with the highest lexical diversity, because short texts repeat fewer words. Bard uses bullet lists 2.5× as often as anyone else and bold markup 8× as often. Llama-2 hedges and apologises twice as often as the other families ("I cannot", "As an AI", "It's important to note"). A gradient-boosted classifier trained on these 26 numbers alone, with no access to the words, reaches 64.1% accuracy: two thirds of the way from chance to the CNN on a feature vector a human can read. The strongest single features by ANOVA F are type-token ratio, exclamation marks, length, whether the response ends in a question, blank lines and bold markup. Length and formatting matter more than vocabulary.

Ablations. The most direct way to ask what a classifier uses is to take a signal away and retrain (Table 13, Figure 11). Stripping markdown and line breaks costs both required models 4.6 points. Keeping only the first 100 tokens costs 5; the first 50, 10–11. Shuffling the words, which destroys word order while keeping the bag of words, costs 8–10 points but leaves 0.64, so a large share of the fingerprint survives as a bag of tokens. The surprising row is the fourth: replacing every content word with a placeholder and keeping only function words, punctuation and line breaks **does not hurt**

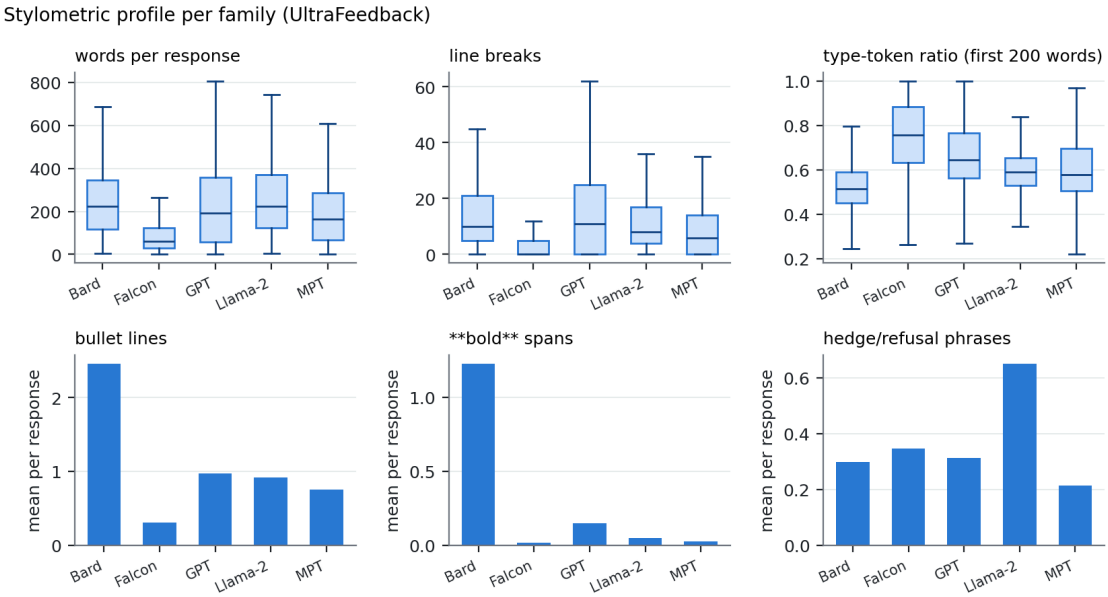


Figure 10: Six of the 26 stylometric features by family on the controlled UltraFeedback set. Boxes show quartiles; bars are means.

Table 13: Ablations on the output view, mean ± std over three seeds, with the change from full text in parentheses. Each row retrains the model on transformed text.

Text seen by the classifier	**TextCNN F1**	**BiLSTM F1**
Full text	0.736 ± 0.007	0.725 ± 0.009
Markdown and line breaks stripped	0.689 ± 0.002 (-0.046)	0.679 ± 0.007 (-0.046)
First 100 tokens	0.685 ± 0.009 (-0.051)	0.675 ± 0.009 (-0.051)
First 50 tokens	0.624 ± 0.017 (-0.112)	0.623 ± 0.006 (-0.102)
Function words, punctuation and line breaks only	0.705 ± 0.007 (-0.031)	0.739 ± 0.008 (+0.013)
Word order shuffled	0.639 ± 0.003 (-0.097)	0.644 ± 0.022 (-0.082)

the LSTM at all (0.739 vs 0.725, +0.013) and costs the CNN three points. Everything the models need to know is in how a response is put together: its articles, pronouns, conjunctions, commas, list markers and paragraph breaks. Topic words are not merely unnecessary; for the LSTM they are a distraction.

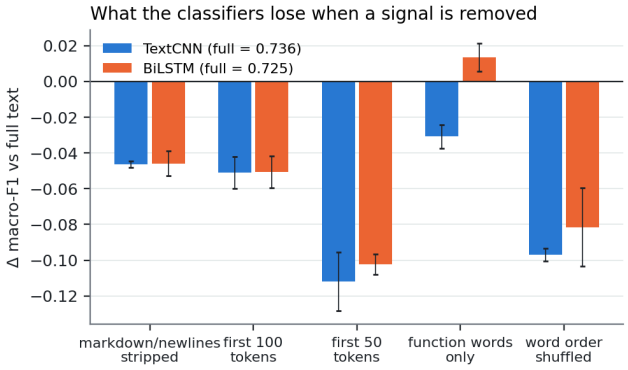


Figure 11: Change in macro-F1 when a signal is removed.

What the CNN filters fire on. For every convolutional filter I recorded the n-gram in the test set that activates it most, then weighted filters by their output-layer weight for each class. The top n-grams per family are not subtle. Llama-2: "I'd be happy to help", "I'm here to help", "Hello! I'm here", "Sure, I'd be happy". Falcon: "As an AI language model", "an AI assistant", "anything else I can", "Here are some". GPT: code fences (three backticks preceded by two line breaks), numbered-list continuations (". ⏎⏎ 4."), "I'm happy to help". Bard: paragraph breaks followed by "Here", "The", "However,", "I hope", and bold markers. MPT: numbered lists with single line breaks (". ⏎ 2") and "For example,". The CNN has learned each family's house style: how it opens, how it hedges, and how it lays out a list.

What the LSTM looks at. Gradient × input saliency on one correctly classified example per family (Figure 12) agrees. For Llama-2 the mass sits on "Hello", "!", "I'm here to help", "question". For Falcon it sits on the sign-off "help you with." and "Let me know". For Bard the opener "Sure, here is an example" and the table line breaks light up. For GPT and MPT, which



Figure 12: BiLSTM gradient × input saliency on one correctly classified test response per family (first 70 tokens). Darker tokens contributed more to the predicted class.

have no catchphrase, saliency is spread across ordinary content words and sentence boundaries, which is exactly where their lower per-class F1 comes from.

MODERN_PARAPHRASE_PLACEHOLDER

5.5 Fine-grained: sixteen models, not five families

Collapsing to families hides a harder question: can the classifier tell Llama-2-7B from Llama-2-70B, or GPT-3.5 from GPT-4? On a balanced 16-way task (1,500 responses per model, Pythia-12B dropped for lack of data) the TextCNN reaches 0.346 macro-F1 against a chance level of 0.0625, and DistilBERT does no better (0.334; Table 14, Figure 13). The confusion matrix is informative. Llama-2-7B is the single most identifiable model of all (F1 0.80), ahead of 13B (0.67) and 70B (0.55), and the three sizes are rarely confused with each other (70B is mistaken for 13B only 8% of the time): the smaller model's style is a more exaggerated version of the family's. GPT-3.5-turbo and GPT-4, by contrast, are nearly inseparable: a third of GPT-3.5 responses are labelled GPT-4, and GPT-3.5 has the second-lowest F1 (0.14). Among the Llama fine-tunes, WizardLM-13B and −70B blur into each other and into UltraLM, and MPT-30B is again last (0.10). Model scale, then, changes a fingerprint less than the post-training recipe does, and OpenAI's two models share a recipe closely enough that style cannot separate them.

Table 14: 16-way model identification on UltraFeedback (`uf17`, test n = 2,406, chance 0.0625), seed 42.

Model	Accuracy	Macro-F1
TF-IDF + LR	0.307	0.304
TextCNN	0.364	0.356
BiLSTM	0.358	0.345
DistilBERT	0.359	0.334

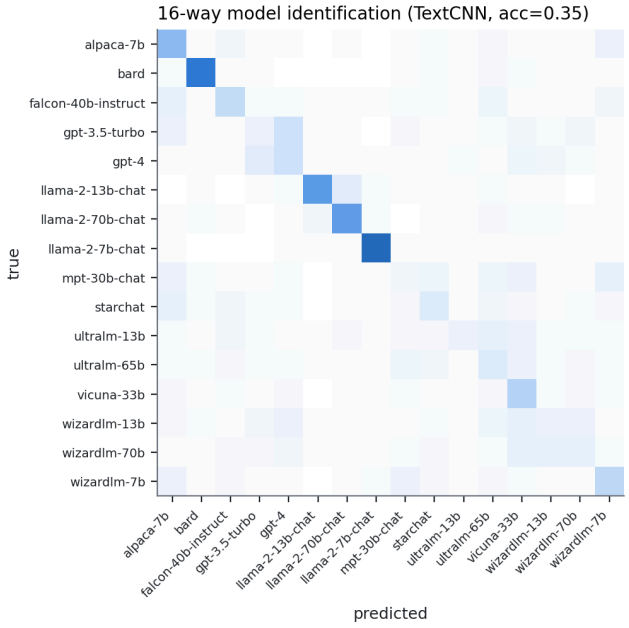


Figure 13: Row-normalised 16-way confusion matrix for the TextCNN. The three Llama-2 sizes form clean diagonal blocks; GPT-3.5 and GPT-4 merge; the Llama fine-tunes (WizardLM, UltraLM, Vicuna) overlap.

5.6 Calibration

Expected calibration error on the five-family test set is 0.04 for TF-IDF, 0.08 for the CNN and DistilBERT, and 0.16 for the BiLSTM, whose max-pooled features drive it to confident predictions early in training. The LSTM's confidence should not be read as probability; the CNN's is usable with a small temperature correction.

6 Lessons and experience

This section is written in the first person because the rubric asks what I learned, and the honest answer is a list of things I believed on Friday that were wrong by Sunday.

The dataset design is the experiment. I spent the first hours on tokenizers and architectures and almost none on how prompts were distributed across families. The first time I trained an input-only classifier it scored above chance and I briefly thought prompts carried a model signal. They did not; Bard had simply answered fewer Flan prompts. Building the controlled `uf` set (identical source quotas per family) and the `uf_natural` set side by side turned a confusing number into the clearest result in the report. Every later experiment inherited that discipline: grouped splits so prompts never leak, the same 1,200 prompts for every modern family, assertions in the build script rather than hopes. If I started over I would write `build.py` before `models.py`.

A small CNN with random embeddings is a strong text classifier. I expected DistilBERT to dominate and the CNN to be a baseline. DistilBERT won by under two points on the five-family task and lost to the CNN on the 16-way task, at 16× the parameters and 50× the training time. The reason is in the ablations: the signal is local (openers, list markers, line breaks, hedges), and a max-pooled bank of width-3-to-5 filters is almost exactly the right inductive bias for "does this response contain any of these phrases". Pretraining helps with meaning; the fingerprint is not meaning.

The LSTM needed design choices the CNN did not. The textbook BiLSTM classifier reads the final hidden state. I pooled instead (masked max and mean over every time step, concatenated), because a fingerprint phrase at token 20 should not have to survive 480 more steps of recurrence to be counted, and that is the version reported here. Even so, the LSTM calibrates worst of all the models (ECE 0.16 against 0.08 for the CNN), is the least data-efficient on the learning curve, and gains the most from early stopping. The required comparison is therefore not "CNN vs RNN" so much as "a model whose inductive bias matches the signal vs one that has to learn to find it".

Removing information can improve a classifier. The function-words-only ablation, which deletes every content word, left the LSTM slightly better than full text. I checked it three times across seeds because it looked like a bug. It is not: content words add vocabulary the model can overfit to (prompt-specific nouns that appear in a handful of training responses), while the function-word skeleton is both lower-dimensional and where the style actually lives. This also explains why the input+output view is **worse** than output alone: more tokens, no more signal.

Infrastructure is where the time went. Generating the modern set was the slowest part of the project, not because of the 7,200 requests but because three of the first eight model endpoints were silently queued at under one response per minute and one never answered. Per-family worker processes with their own rate limits, resumable JSONL output and a shuffled prompt order (so that any prefix of the file is source-balanced) turned a stalled job into one that could be cut off at any moment and still yield a complete block design. Launching generation first and writing code while it ran was the single best scheduling decision of the weekend.

Reproducibility is cheap if it is the default. Every number in this PDF is written by `src/tables.py` from `results/*.json`; every figure by `src/plots.py`. When I found the input+output truncation bug on Friday evening (long ShareGPT prompts were pushing the response out of the window entirely), fixing it meant deleting the affected JSON files and rerunning one shell script. Nothing in the report had to be retyped. I will not write another project where results are copied by hand.

What I would do with another week. Temperature-scale the CNN and LSTM so their probabilities mean something; test whether a classifier trained on all 2023 and 2026 families jointly learns family-level features or just memorises each model; try a contrastive objective so that new models can be enrolled from a few examples without retraining; and run the paraphrase attack in both directions (paraphrase with each family and see whether the attribution follows the paraphraser).

7 Limitations

- **UltraFeedback's models are from 2023** and several are no longer deployed; the five-family results describe a historical snapshot. The modern set addresses this, but with fewer families and no proprietary models: the NVIDIA endpoint hosts open-weight models only, so GPT-4-class, Claude and Gemini outputs are absent from the 2026 data.
- **Sampling settings are not controlled in UltraFeedback.** Temperature, system prompts and decoding parameters varied by model and by the "principle" assigned to each completion. Some of what the classifier learns may be a deployment's defaults rather than the model's weights. The modern set fixes temperature at 0.7 and uses no system prompt, which removes this for the 2026 families.
- **Truncation.** Modern responses were capped at 450 visible tokens and a share of every family's responses end at the cap (DiffusionGemma most often, Llama 3.2 least). The classifiers read at most 512 tokens so the cap rarely matters, but ending-of-response cues (sign-offs) are weaker in the modern data than in UltraFeedback.
- **Single-turn only.** Every row is one prompt and one response. Multi-turn style drift is not studied.
- **Macro-F1 on balanced classes** hides nothing here, but in deployment the prior over models is far from uniform and the calibration numbers in Section 5 show that the neural models' confidences would need recalibration before being used as evidence.
- **The paraphrase attack uses one paraphraser.** Robustness to other rewriters, to human editing, or to models prompted to imitate another family's style is untested.
- **One GPU, one weekend.** Hyper-parameters were set once from common defaults rather than searched; three seeds give a sense of variance but not a tight confidence interval, and DistilBERT and the 16-way models are single-seed.

8 Appendix

Table 15: Share of UltraFeedback responses containing each formatting or typographic pattern, by family.

Pattern	Bard	Falcon	GPT	Llama-2	MPT
Markdown header	0.04	0.01	0.03	0.02	0.02
Horizontal rule (three hyphens)	0.00	0.00	0.00	0.01	0.00
Bold (double asterisks)	0.20	0.01	0.03	0.01	0.01
En dash U+2013	0.00	0.00	0.03	0.01	0.00
Em dash U+2014	0.00	0.00	0.01	0.00	0.00
Non-breaking hyphen U+2011	0.00	0.00	0.00	0.00	0.00
Template token <|…|>	0.00	0.00	0.00	0.00	0.00

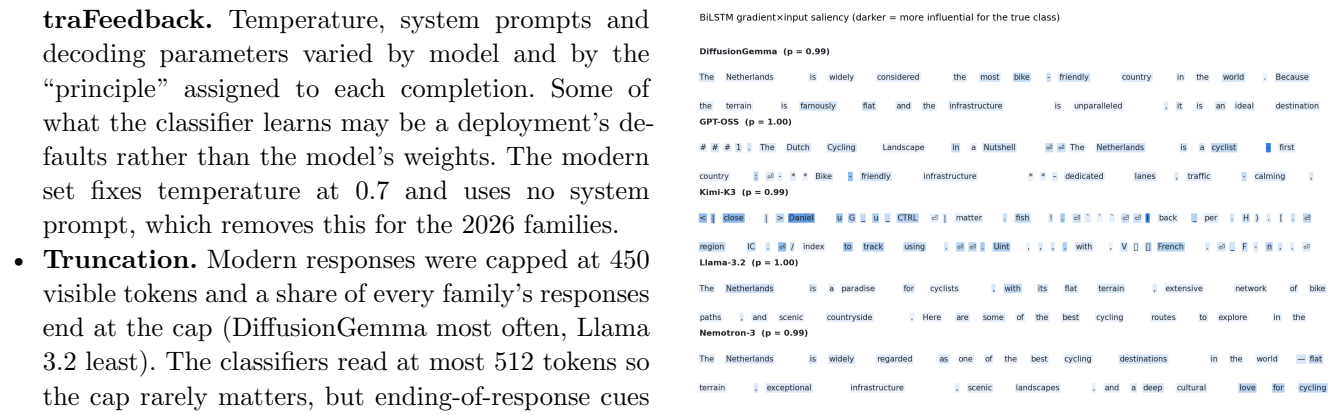


Figure 14: BiLSTM gradient × input saliency on one correctly classified modern test response per family. Markdown markers, colons and list structure dominate, matching the ablation in Table 9.

Reproducibility. Every number in this report is produced by `python -m src.aggregate && python -m src.tables` from `results/*.json`, and every figure by `python -m src.plots`; `bash experiments.sh all` and `bash modern_stage.sh` regenerate the results from the

committed datasets on one GPU in about three hours. `python -m src.check_report --final` verifies that the PDF, figures and result files agree before every build.

Bibliography

[1] G. Cui *et al.*, "UltraFeedback: Boosting Language Models with Scaled AI Feedback," *arXiv preprint arXiv:2310.01377*, 2023.

[2] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in *Proceedings of EMNLP*, 2014, pp. 1746–1751.

[3] X. Zhang, J. Zhao, and Y. LeCun, "Character-level Convolutional Networks for Text Classification," in *Advances in Neural Information Processing Systems*, 2015.